Papra Architecture Guide

Nov 3, 2026 · @Abdullah Al Rashid

Papra is a pnpm monorepo (commit `93a569f`) whose core is one Hono + Drizzle/libSQL server (`apps/papra-server`) and one SolidJS client (`apps/papra-client`), with shared libraries under `packages/`.

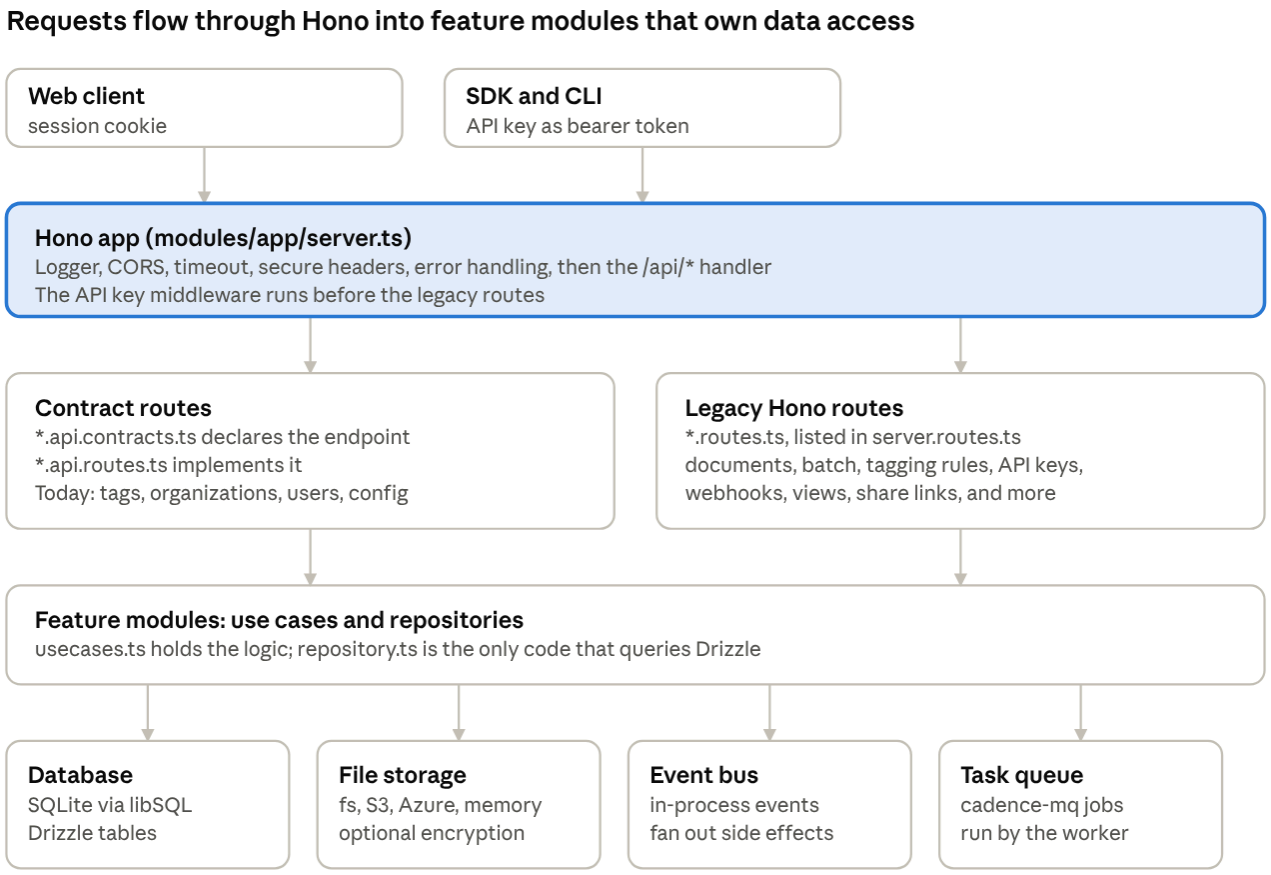
Repo map

Path	What it is
`apps/papra-server`	The API server and background worker (Hono, Drizzle ORM, libSQL, better-auth, valibot, cadence-mq task queue)
`apps/papra-client`	The web UI, organised by feature module (`documents`, `tags`, `tagging-rules`, `organizations`, …)
`apps/docs`, `apps/website`	Documentation site and marketing site
`apps/mobile`	Mobile app
`packages/api-sdk`	Typed HTTP SDK: a contract-based client plus a legacy client
`packages/app`	Docker packaging of the application (Dockerfiles, README)
`packages/cli`	Command line interface for the API
`packages/lecture`	Text extraction from files (used for document content)
`packages/search-parser`	Parser for the search query language (filters, AND/OR, grouping)
`packages/webhooks`	Webhook signing and verification helper
`packages/std`	Small shared utilities

Tooling: `pnpm-workspace.yaml` lists `packages/*` and `apps/*`; lint and format use oxlint and oxfmt; tests run with vitest from the root.

Server module layout

The server is a feature-sliced modular monolith: every domain lives in `apps/papra-server/src/modules/<feature>/`, and files are named `<feature>.<role>.ts`. Wiring is explicit dependency injection, with no container.



server request path · 2 route systems, 4 backing services

Every request enters Hono, is answered by a contract route when one matches, and otherwise falls through to a legacy route; both call the same use case and repository layer.

Anatomy of a feature module. The roles recur across features (see `tags/`, `documents/`, `tagging-rules/`):

- `*.table.ts` / `*.tables.ts`: Drizzle table definitions
- `*.repository.ts`: the only layer that runs queries (a `create<Feature>Repository({ db })` factory); `*.repository.models.ts` maps rows to domain objects
- `*.usecases.ts`: business logic, built as `create<Feature>Usecases(deps)`
- `*.routes.ts` (legacy Hono) or `*.api.contracts.ts` + `*.api.routes.ts` (new contract system)
- `*.schemas.ts`: valibot input validation; `*.types.ts`, `*.constants.ts`, `*.errors.ts`, `*.config.ts`

- `events/` and `tasks/` subfolders for event handlers and background jobs

Boot sequence. `src/start.ts` loads config, opens the database, runs migrations, and builds one `GlobalDependencies` object (`config`, `db`, `auth`, `documentsStorageService`, `documentSearchServices`, `eventServices`, `taskServices`, `webhookTriggerServices`, `kvStore`, `aiServices`, and others, typed in `modules/app/server.types.ts`). That object is passed into every route registrar and task registrar. `processMode` (`web`, `worker`, or `all`, from `modules/app/process.models.ts`) decides whether to start the HTTP server, the task worker plus ingestion-folder watcher, or both.

Request path. `modules/app/server.ts` builds the Hono app with logger, CORS, timeout, secure-headers and error middleware, then static assets. Next, `registerApiRoutes` mounts `/api/*` through the new contract request handler. Anything it does not claim falls through to the API-key middleware and the legacy `registerRoutes` list in `server.routes.ts`, where each feature registers its own routes.

Cross-cutting modules. `app/` (server, auth via better-auth, database, events, middlewares), `api/` (contract engine), `config/`, `storage/` (drivers: fs, memory, S3, Azure Blob; plus encryption), `tasks/` (cadence-mq queue), `kv-store/`, `emails/`, `webhooks/`, `ai/` and `ai-credits/`, `subscriptions/`, `plans/`, `plan-entitlements/`, `roles/`, and `shared/` (logger, db helpers, errors, crypto, streams and similar utilities).

Domain modules. `organizations/` (the tenant boundary), `users/`, `documents/` (with sub-modules for content extraction, search, activity, batch, storage patterns, encryption), `tags/`, `tagging-rules/`, `auto-tagging/`, `document-enrichment/`, `custom-properties/`, `document-views/`, `document-share-links/`, `intake-emails/`, `ingestion-folders/`, `invitations/`, `api-keys/`, and `admin/`.

Events. Use cases emit domain events (for example `document.created`, `document.tags-changed`) on an in-process event bus (`modules/app/events`). Handlers registered in `events.handlers.ts` write the activity log, sync the search index, fire webhooks, and track analytics, so feature code stays decoupled from those side effects.

Database schema

The schema is split across feature modules, not held in one file. Each module declares its own Drizzle tables in `*.table.ts` or `*.tables.ts` next to its repository, and `drizzle.config.ts` collects them with the glob `./src/modules/**/*.table(s).ts`. The database is SQLite through libSQL (`dialect: 'turso'`, default `file:./db.sqlite`).

Migrations are hand-written, not generated. Despite the drizzle-kit config, the schema is changed through TypeScript migrations in `apps/papra-server/src/migrations/list/NNNN-name.migration.ts` (28 as of this commit). Each

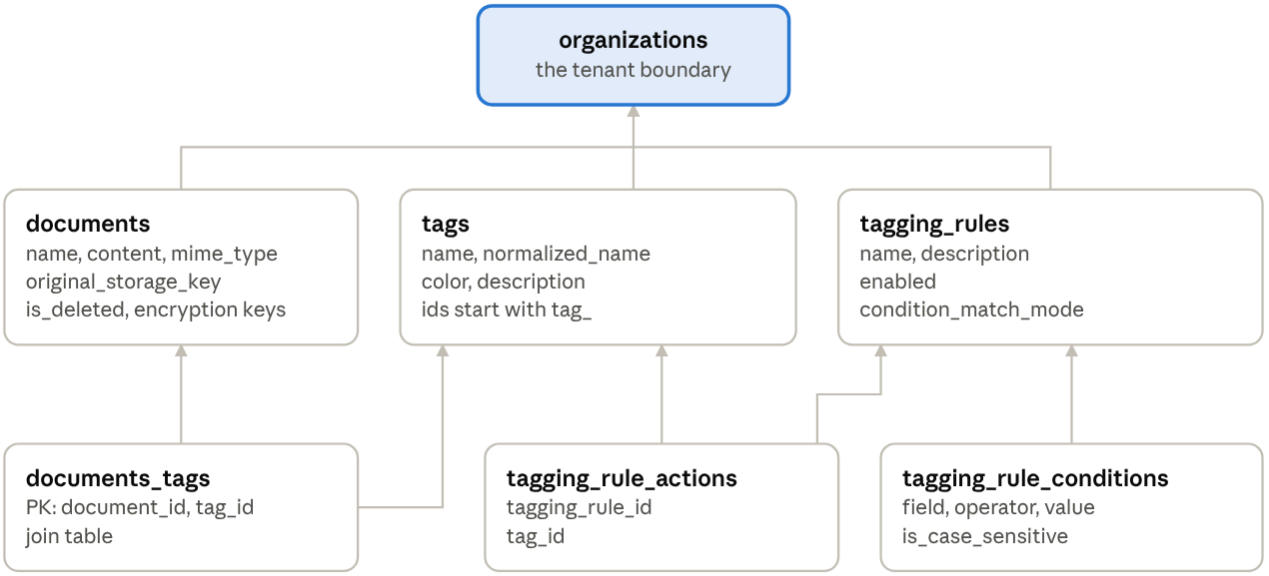
exports a `Migration` (`name` , optional `description` , `up` , optional `down`) and is added to `migrations/migrations.registry.ts` . `migrations.usecases.ts` runs the ones not yet recorded in a migrations table. Scripts: `pnpm migrate:up` , `migrate:down` , `migrate:create` , and `start:with-migrations` for production. Migration 0001 creates the base tables with raw SQL.

Conventions. `shared/db/columns.helpers.ts` provides `createPrimaryKeyField` (text ids with a per-entity prefix such as `tag_`) and `createTimestampColumns` (`created_at` , `updated_at` , epoch milliseconds). Every domain table references `organizations.id` with `ON DELETE CASCADE` , which makes the organization the tenant boundary.

Where the main tables live

Tables	File
`users` , `organizations` , `organization_members` , `organization_invitations`	`users/users.table.ts` , `organizations/organizations.table.ts`
auth sessions and accounts (better-auth)	`app/auth/auth.tables.ts`
`documents`	`documents/documents.table.ts`
`documents_fts` (full-text index)	`documents/document-search/database-fts5/database-fts5.tables.ts`
`tags` , `documents_tags`	`tags/tags.table.ts`
`tagging_rules` , `tagging_rule_conditions` , `tagging_rule_actions`	`tagging-rules/tagging-rules.tables.ts`
document activity log	`documents/document-activity/document-activity.table.ts`
custom properties and values	`custom-properties/`
views, share links, intake emails, API keys, webhooks, subscriptions, plans, AI credits, KV store	each in its own module folder

Tags join documents through documents_tags, all under one organization



Arrows point at the referenced table. Deleting a parent row cascades to the rows that reference it.

documents and tags schema · 7 tables

The tag tables are the part of the schema most connected to documents: `documents_tags` is the only bridge, and rules reach documents only through the tags they apply.

How the client talks to the API

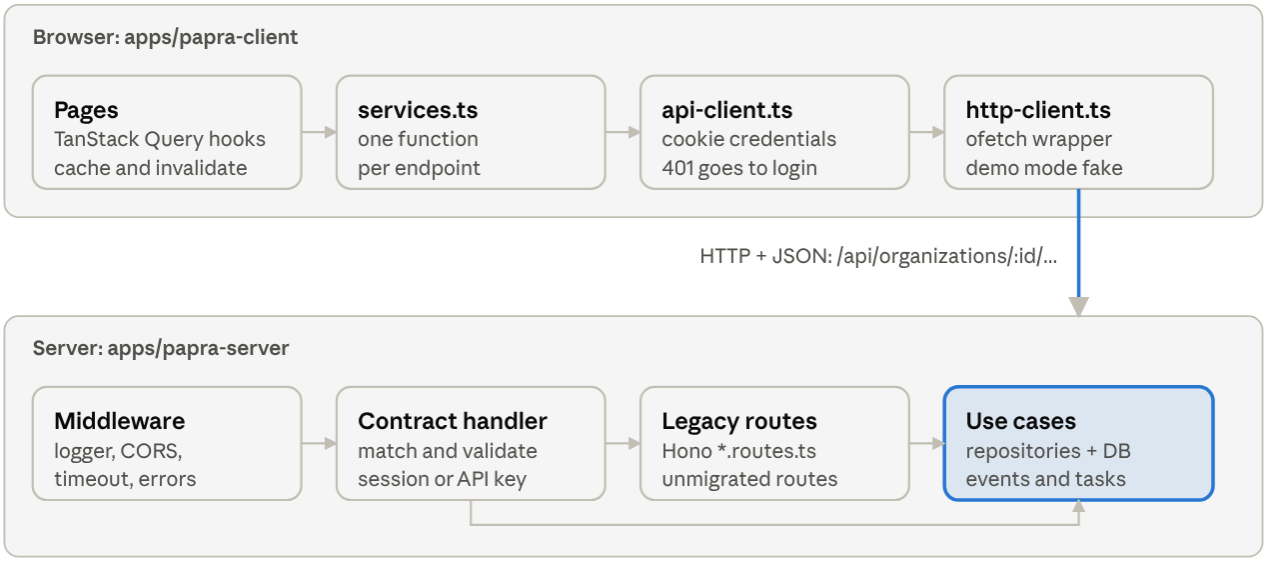
The web client calls the server with plain JSON over HTTP, one thin service function per endpoint, cached by TanStack Query. It does not use the SDK package; it hand-writes paths that match the server's routes.

Client layers (all under `apps/papra-client/src/modules/`):

1. `<feature>/<feature>.services.ts` holds one async function per endpoint, for example `fetchTags`, `createTag`, `addTagToDocument` in `tags/tags.services.ts`, and `uploadDocument`, `fetchOrganizationDocuments` in `documents/documents.services.ts`.
2. `shared/http/api-client.ts` wraps every call: it sets `baseUrl` to `VITE_BASE_API_URL` (default: the page's own origin), sends `credentials: 'include'` so the session cookie travels, and redirects to the login page, keeping the current path, on a 401.
3. `shared/http/http-client.ts` is a thin `ofetch` wrapper. In demo mode it swaps in an in-browser fake (`modules/demo`).

4. Pages and components call these services through `useQuery` / `useMutation` from `@tanstack/solid-query`, with keys like `['organizations', orgId, 'tags']`. After a mutation they invalidate that key.
5. Responses come back as DTOs with ISO date strings; `coerceDates` turns them into `Date` objects. Uploads use `FormData` via `getFormData`.

The client sends cookie-authenticated JSON calls to one of two route systems



The contract handler answers first; requests it does not claim fall through to the legacy routes.

client to API call path · 4 client layers, 4 server stages

The browser reaches the server in one hop; the only routing decision, contract handler or legacy route, happens on the server and is invisible to the client.

Server side. All routes are tenant-scoped: `/api/organizations/:organizationId/...`. Authentication is resolved per route: a better-auth session cookie (what the web client uses) or an API key sent as `Authorization: Bearer <token>` (what the SDK and CLI use). API-key routes declare the permissions they need, such as `documents:update` and `tags:read` for adding a tag to a document. Each handler then calls `ensureUserIsInOrganization` before touching data.

Two route systems coexist during a migration. Per `modules/api/README.md`, since October 2026 routes are defined as contracts: `*.api.contracts.ts` declares method, path, valibot request schemas, and responses; `*.api.routes.ts` implements them; `api.contracts.registry.ts` exports the `apiContracts` collection. `packages/api-sdk` builds a typed client from those contracts (`createClient()` merges a contract client with a `legacy-client`). Not yet migrated routes remain in `*.routes.ts` (Hono). Tags, organizations, users, and config are on contracts today. Documents, tagging rules, and batch operations are still legacy. The client's paths work with both because the URLs are identical.

Documents deep dive

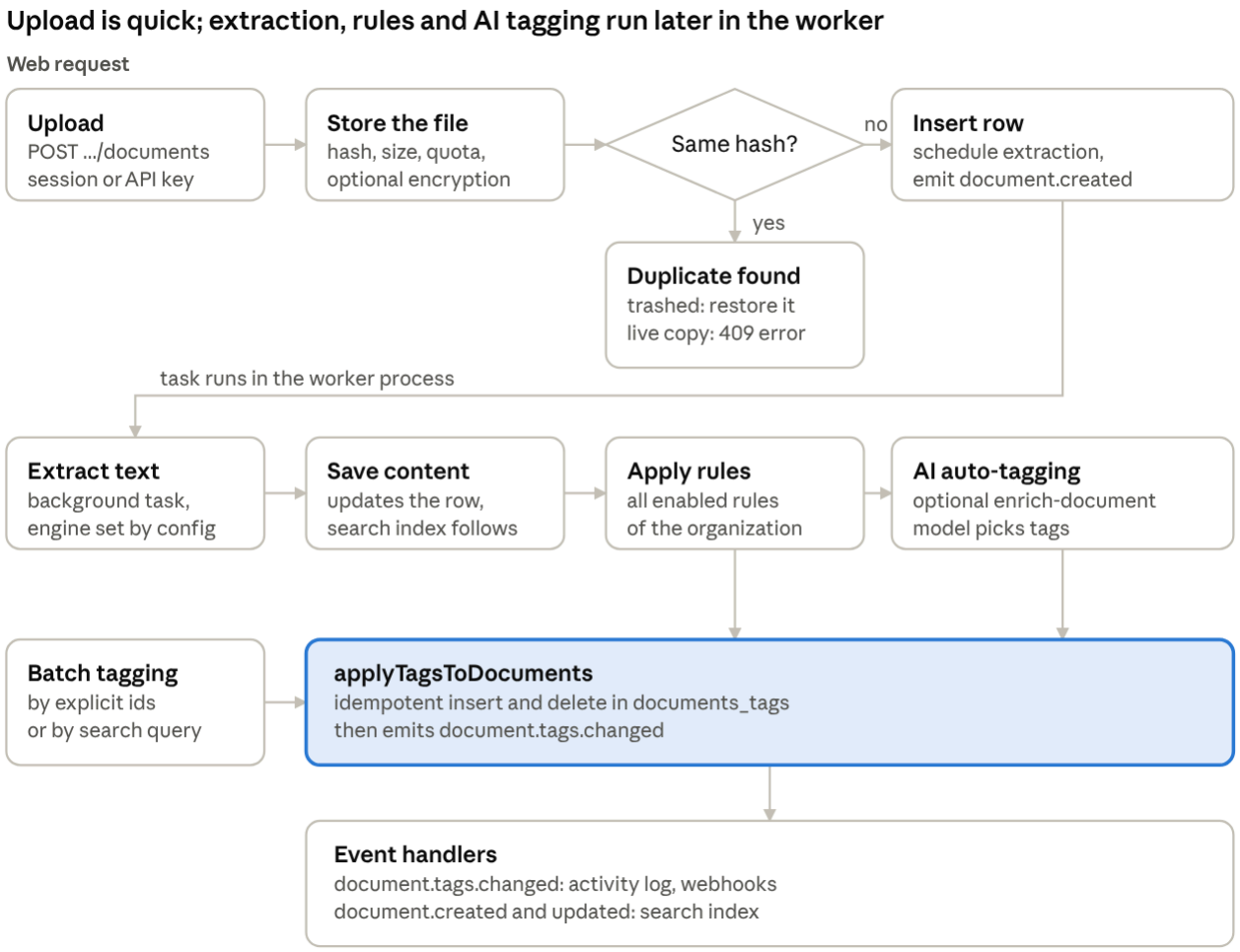
A document is a file in blob storage plus one row in `documents`; everything after the upload (text extraction, tagging, indexing, webhooks) is driven by an event and a background task, not by the upload request itself.

The record. `documents` (`documents.table.ts`) stores `organizationId`, `createdBy`, the original file facts (`originalName`, `originalSize`, `originalStorageKey`, `originalSha256Hash`), a mutable `name`, `mimeType`, extracted `content`, `documentDate`, `notes`, optional encryption metadata (wrapped key, KEK version, algorithm), and soft-delete columns (`isDeleted`, `deletedAt`, `deletedBy`). The bytes live outside the database, in the storage driver selected by config.

Upload path (`POST /api/organizations/:organizationId/documents`, legacy route in `documents.routes.ts`, logic in `createDocument` in `documents.usecases.ts`):

1. Authenticate (session, or API key with `documents:create`), check organization membership, and read the plan's `maxFileSize`.
2. Parse the multipart body as a stream. A single pass computes the SHA-256 hash and counts bytes, aborting if the size or the remaining organization storage quota is exceeded, while the file is saved through `documentsStorageService` (optionally encrypted). The storage key comes from the configurable storage pattern.
3. Deduplicate by hash within the organization. If the same hash exists and is not deleted, the new file is removed and the upload fails with a document-already-exists error. If it exists in the trash, the document is restored with the new name, its old tags are cleared, and tagging rules are re-applied.
4. Otherwise re-check the quota, insert the row, and schedule the `extract-document-file-content` task. If the insert fails, the stored file is deleted.
5. Emit `document.created`.

The same `createDocumentCreationUsecase` also serves the other intake paths: intake emails and the ingestion-folder watcher.



document upload and tagging pipeline · web request, worker, shared tag writer

The request only stores the file and writes the row. Text extraction, tagging rules, and AI tagging follow in the worker, and every automatic or batch tag change passes through `applyTagsToDocuments`. Single-document tag and untag calls use `addTagToDocument` and a direct removal instead, and emit the activity-log and webhook side effects themselves.

Background processing (worker mode). `extract-document-file-content` reads the file back from storage and extracts text through a strategy chosen by config (built-in `lecture` package, Docling, Mistral OCR, Azure DI, or a custom HTTP service). It saves `content`, which emits `document.updated`, then runs the organization's tagging rules. If AI and auto-tagging are enabled, it schedules `enrich-document` (see Tags). Other tasks: `hard-delete-expired-documents` purges trashed documents after retention.

Events fan out side effects. Handlers in `events.handlers.ts` index and update the search index (`documents_fts`, SQLite FTS5, behind a pluggable search provider), write the activity log, trigger webhooks, track analytics, and keep the storage key in sync when a document is renamed.

Read and change paths. `GET .../documents` runs a search query through the FTS5 provider, then `enrichAndFormatDocumentsForApi` attaches tags and custom properties in two batched queries. Other routes: `GET .../documents/:id`, `GET .../documents/:id/file` (streams and decrypts), `PATCH` (rename, notes, date), `DELETE` (soft delete to trash), `POST .../restore`, `POST .../reprocess`, trash listing and purge, and statistics. Batch routes (`documents-batch`) trash or tag many documents at once, addressed either by explicit ids (up to a cap) or by a search query.

Client side. `documents.services.ts` holds `uploadDocument` (FormData), `fetchOrganizationDocuments`, `fetchDocument`, `fetchDocumentFile` (blob), `deleteDocument`, `restoreDocument`, `reprocessDocument`, and the trash calls. Pages live in `documents/pages/`.

Tags deep dive

Tags are organization-scoped labels attached to documents through a join table. Three things add or remove them: people (single or batch), tagging rules, and AI auto-tagging. All three end in the same function, so every change emits the same event.

Schema (`tags/tags.table.ts`, `tagging-rules/tagging-rules.tables.ts`):

Table	Key columns
`tags`	`id` (prefix `tag`), `organization_id` (cascade), `name`, `normalized_name`, `color` (hex), `description`, timestamps
`documents_tags`	composite primary key (`document_id`, `tag_id`), both cascade on delete
`tagging_rules`	`organization_id`, `name`, `description`, `enabled`, `condition_match_mode` (`all` or `any`)
`tagging_rule_conditions`	`tagging_rule_id`, `field` (`name` or `content`), `operator`, `value`, `is_case_sensitive`
`tagging_rule_actions`	`tagging_rule_id`, `tag_id`

`normalized_name` is the lowercased name (`normalizeTagName`); it makes names unique per organization regardless of case and powers tag lookup in search. Migration `0016` backfilled it and de-duplicated colliding names. Deleting a tag or document cascades through `documents_tags` and rule actions.

Direct tag management (`tags.api.contracts.ts`, `tags.api.routes.ts`, already on the new contract system):

Operation	Route	Notes
List tags	`GET /api/organizations/:orgId/tags`	Returns each tag with a count of its non-deleted documents
Create	`POST .../tags`	Name 1 to 64 characters, color `#RRGGBB`, optional description up to 256; capped at 200 tags per organization (`MAX_TAGS_PER_ORGANIZATION`)
Update, delete	`PUT`, `DELETE .../tags/:tagId`	Not-found error if the tag belongs to another organization
Tag a document	`POST .../documents/:documentId/tags` with `{ tagId }`	204; 409 `documents.already_has_tag`; needs API-key permissions `documents:update` and `tags:read`
Untag a document	`DELETE .../documents/:documentId/tags/:tagId`	204
Batch tag	`POST .../documents/batch/tags`	Body: `filter` (explicit `documentIds`, or a search `query`), `addTagIds`, `removeTagIds`. Legacy route, kept ahead of the contract router for path precedence

Tagging a single document goes through `addTagToDocument` (insert, `document:tag:added` webhook, `tagged` activity-log entry). Batch and automatic tagging go through `applyTagsToDocuments`, which validates every tag against the organization, inserts and deletes pairs in bulk and idempotently, and emits one `document.tags.changed` event listing the pairs that really changed. Event handlers then write `tagged` and `untagged` activity entries and fire `document:tag:*` webhooks.

Tagging rules (`tagging-rules/`, legacy `*.routes.ts` under `.../tagging-rules`, including `POST .../:taggingRuleId/apply`). A rule is a set of conditions plus a set of tags to apply. Conditions compare a document field (`name` or `content`) with one operator (`equal`, `not_equal`, `contains`, `not_contains`, `starts_with`, `ends_with`), optionally case-sensitive. `all` mode requires every condition; `any` requires one; a rule with no conditions always matches. Evaluation happens at three points:

1. After text extraction, because rules usually match on `content`: `applyTaggingRules` runs every enabled rule of the organization against the document.
2. When a trashed duplicate is re-uploaded and restored.
3. On demand: `apply` schedules the `apply-tagging-rule-to-documents` task, which walks the organization's existing documents. A faulty rule only logs an error and never breaks the flow that triggered it.

AI auto-tagging (`auto-tagging/`, `document-enrichment/`). When AI and auto-tagging are enabled in config, the extraction task schedules `enrich-document`. It skips if the organization disabled auto-tagging, or has no tags and may not create new ones. Otherwise it prompts a model with the document and the existing tags, receives tag ids to add and optionally names to create (limit set by `maxTags`; new tags get color `#CCCCCC`), then applies them with `applyTagsToDocuments`.

Read side. Document listings and detail attach `tags` through `getTagsByDocumentIds`. Search supports tag filters by id or by name (`normalized_name`), joined through `documents_tags` and scoped to the organization, so batch actions can target a query such as everything carrying a tag.

Client side (`apps/papra-client/src/modules/tags`, `tagging-rules`). `tags.services.ts` mirrors the routes above (`fetchTags`, `createTag`, `updateTag`, `deleteTag`, `addTagToDocument`, `removeTagFromDocument`). `tags.page.tsx` queries `['organizations', orgId, 'tags']` and invalidates it on mutation. Reusable components include `tag-picker`, `tag-list`, and `tag`, and `tagging-rules.services.ts` covers rule CRUD and apply.